

Security of Approximate Neural Networks against Power Side-channel Attacks

Aditya Japa^{*}, Jack Miskelly^{**}, Maire O'Neill^{**}, Chongyan Gu^{**}

^{*}*School of Computing, Engineering and Intelligent Systems, Ulster University, Derry, U.K.*

^{**}*Centre for Secure Information Technologies, Queen's University Belfast, Belfast, U.K.*

Email: a.japa@ulster.ac.uk, and c.gu@qub.ac.uk

Abstract—Emerging low-energy computing technologies, in particular approximate computing, are becoming increasingly relevant in key applications. A significant use case for these technologies is reduced energy consumption in Artificial Neural Networks (ANNs), an increasingly pressing concern with the rapid growth of AI deployments. It is essential we understand the security implications of approximate computing in an ANN context before this practice becomes commonplace. In this work, we examine the test case of approximate ANN processing elements (PE) in terms of information leakage via the power side channel. We perform a weight extraction correlation Power Analysis (CPA) attack under three approximation scenarios: overclocking, voltage scaling, and circuit level bitwise approximation. We demonstrate that as the degree of approximation increases the Signal to Noise Ratio (SNR) of power traces rapidly degrades. We show that the Measurement to Disclosure (MTD) increases for all approximate techniques. An MTD of 48 under precise computing is increased to at minimum 200 (bitwise approximate circuit at 25% approximation), and under some approximation scenarios >1024 . i.e. an increase in attack difficulty of at least $\times 4$ and potentially over $\times 20$. A relative Security-Power-Delay (SPD) analysis reveals that, in addition to the across the board improvement vs precise computing, voltage and clock scaling both significantly outperform approximate circuits with voltage scaling as the highest performing technique.

Index Terms—Neural network hardware, Power Side-channel attacks, Approximate computing, Artificial intelligence

I. INTRODUCTION

Driven by the increasing global demand for high performance centralized compute resources, the total energy demands of datacentres and compute clusters has been growing at a breakneck pace in recent years. One of the key factors in this has been the sudden explosion of computationally intensive ANN based AI applications. For both environmental and economic reasons there is now a strong drive towards more energy efficient computing hardware in this context. One of the key techniques for low-energy computing is approximate computing (sometimes also called transprecise or inexact computing). The approximate computing paradigm, in essence, reduces energy consumption by removing the constraint that every output bit of a computation must be correct. Instead a controlled and tolerable degree of error is permitted. This, in turn, allows for increases in energy efficiency or throughput relative to an exact circuit. The corollary is that to take full advantage of approximate computing requires an error-tolerant

application. ANNs, which possess an inherent error tolerance, are therefore one of the areas where approximation will have the widest impact.

When it comes to the secure information in ANNs there are two major areas of concern. The first is the information being processed by the network which in itself may be sensitive, especially as deployments become more common in areas like health and finance. The second are the weights and structure of the network itself. Model training is a high effort and resource intensive task. This makes the results of training, especially the weights, commercially sensitive. If an adversary can acquire them they can in effect steal the model, skipping the laborious training step but gaining all the benefits of it. In both cases there are clear incentives for attackers. Extracting this information as a purely remote attack is challenging. However, this changes when considering attackers with Side Channel Analysis (SCA) capability. In brief, SCA involves the analysis of characteristics such as power consumption, electromagnetic radiation emissions, and timing which are not intended to carry information but which nonetheless respond to operations in the system. Power draw is of particular significance in an ANN context, because there is a strong correlation between node weights, inputs, and power consumption.

SCA may seem like a difficult thing to exploit against the largely centralized compute clusters used for ANNs currently, but this state of affairs is unlikely to last. There is an increasing interest in edge deployments of ANNs to avoid the inherent inefficiencies and latency issues of centralized clusters. The latter point is especially significant for real time applications. Edge deployments inevitably expose devices to physical level attacks like SCA. When we consider that energy budget is even more critical for these systems, the future edge deployment of approximate ANNs (AANNs) seems highly probable. As approximation alters power characteristics, it seems intuitive that it will have an impact on SCA capability. It is essential we understand what exactly this impact will be in advance. While there is a substantial existing body of work on SCA against ANNs, and there has been some investigation of the security of approximate systems in a general sense, so far there has been no close analysis of the intersection of these technologies.

To rectify that, in this work we investigate how approximation of ANN Processing elements impacts their vulnerability to SCA. More specifically, we are considering an attacker with power monitoring capability who aims to extract the

This work was supported by EPSRC (UK) under the Grant EP/X009602/1.

weights from the network and thereby steal the IP of the network owner. The question is, will the approximation help or hinder the weight stealing attack and to what degree? In this work we will demonstrate that approximation significantly reduces the weight information leakage through the power side channel. Further, that the different forms of approximation offer markedly different levels of protection. This provides a valuable insight for future hardware designers aiming to combine these technologies.

The specific contributions are as follows:

- We provide a power analysis of approximate PEs under three approximation scenarios: clock scaling, voltage scaling, and bitwise approximate circuitry.
- We demonstrate, for the first time, that approximation of all types worsens the SNR of power traces from an attacker's perspective and that the rate of change of SNR relative to degree of approximation varies significantly between methods of approximation.
- We demonstrate how this leads to a higher requirement in terms of MTD for weight stealing attacks, with an increase in attack difficulty of at least x4 in the worst case and more than x20 in the best case.
- We provide a Security-Power-Delay (SPD) analysis of the different methods of approximation, which reveals significant differences in relative performance when security is considered.

II. BACKGROUND

An ANN consists of layers of interconnected computational units (artificial neurons) which can also be called nodes. There are typically at least three layers: an input layer, one or more internal hidden layers, and a final output layer as shown in Fig.1. Each node represents a single neuron that produces a weighted sum. The nodes of each layer are connected to the nodes of the next layer with each such connection being assigned a weight. The nodes themselves implement an activation function which takes the full set of individual inputs and their weights and converts them to a single output. The number of nodes in each layer, the number of hidden layers, and the specific activation function used are implementation specific factors. Weights are determined via model training, in which the model is provided with labelled training data and a defined loss function (i.e. a function which quantifies the difference between predicted output and actual output). Weights are iteratively updated over multiple rounds of training until the loss function is sufficiently minimized. Training is resource intensive and may make use of proprietary training data. As such the final trained weights can have significant commercial value. Knowledge of the weights, layer structure, and activation function is sufficient to create an exact duplicate of the original model.

A. ANNs

Key hardware elements for ANNs are the implementation of the activation function for the nodes and the memory used to store the weight values. Activation functions are simply

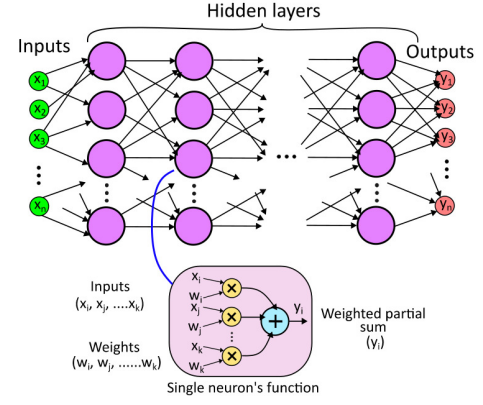


Fig. 1: ANN diagram with different layers

mathematical functions and can be implemented on general purpose hardware, but in accelerated architectures they often receive a dedicated hardware implementation for performance reasons. As the activation function is the primary computation of the network the throughput of the overall network is strongly tied to the throughput of the activation function in each node (the other major factor is the latency of memory accesses). Weights generally must be held in a memory of some kind if the ANN hardware is intended to be adjustable, but the distribution of this memory (per-connection, per-node, per-layer, etc.) can vary. The alternative is to embed the values directly as a physical property of the circuit, but in practice this approach is rare.

B. SCA of ANNs

From a security perspective there are several key areas where side channel leakage must be considered. First, the structure of the network in terms of number of nodes per layer, number of hidden layers, activation function, etc. i.e. the model architecture. These properties are difficult to conceal, and architectural reverse engineering attacks have been demonstrated using system resource allocation/memory access volume [1] [2], cache timing side channel [3], EM side channel [4] and power side channel [5] [6] [7]. This is not a threat limited to simplified cases: Yan et al. demonstrated such an attack using power analysis against complex, non-binarized networks on Nvidia's NVDLA architecture [8]. Knowledge of these properties is advantageous to an adversary in itself, because they both provide a grounding for performing weight-stealing attacks and allow the adversary to greatly reduce the cost of training their own model for a similar task. There is therefore a strong financial incentive for such attacks with or without a weight-stealing follow up attack.

Second, the weights of each connection in the network. If the exact weights can be acquired in addition to the structure being known, the adversary can exactly replicate (i.e. steal) the trained model. Batina et al. performed an EM side channel based attack on ARM based edge devices, which was able to extract not only weights but also the full architectural details [9]. Maji et al. performed a similar weight stealing

attack on ARM, ATmega, and RISC-V based devices using simple power analysis and timing information [10]. Yoshida et al. demonstrated both EM [11] and power analysis [12] attacks against models with encrypted weights implemented on FPGA. Dubey et al. similarly demonstrated the ability to fully extract weights from FPGA using power analysis [13] and proposed masking methods to counter such attacks. This is not a comprehensive list, but it suffices to demonstrate that exact weight extraction is a tangible threat, especially for edge devices.

Third, the data used in training and operation. Attacks which seek to learn the properties of input data based on model output are called model inversion attacks. These have been known in a general sense for almost ten years now [14], and specific proposals have been made which exploit both timing [15] and power [16] side channels to achieve the inversion. Property and membership inference attacks both aim to derive information about the original training data from the model. This form of attack has been demonstrated a number of times in a federated learning context (e.g. [17]–[20]) but the application of side channel analysis in this type of attack remains largely unexplored.

III. EXPERIMENTAL SETUP FOR EVALUATING APPROXIMATE ANNS

A. Approximate Computing in Neural Networks

Approximation as a technique for efficient ANNs is an area of active interest, with upwards of 30 works being published each year at the major design automation conferences alone [21], [22]. This is too broad a field to closely examine in the context of a short paper, but it suffices to know that there are two main forms of approximate computing used in this context: Voltage and Frequency Scaling (VFS) and circuit level bitwise approximation. Frequency scaling involves increasing the operating frequency (overclocking) for increased throughput. Voltage scaling reduces the operating voltage (undervolting) for reduced power consumption. In both cases the tradeoff is a set of semi-random errors at each layer based on the distribution of critical paths (the data paths with the highest propagation delays). In these methods, while the operating characteristics of the overall circuit are changed the hardware of the processing elements are the same as in an exact ANN. In contrast to these methods are bitwise approximate circuits, where the fundamental PEs are redesigned such that specific bits of the operation (the activation function in an ANN context) are approximated, while the remaining bits remain exact. With this method, error distribution is more controlled at the cost of requiring custom hardware. In a general sense, all of these methods can be viewed as increasing efficiency at the cost of introducing noise to the system. The key design choice is what error rate (i.e. how much noise) and distribution can be tolerated without an unacceptably high loss of model accuracy.

The goal of this work is to evaluate the *relative* leakage of different methods of approximation, rather than a specific ANN hardware implementation. As such, we have selected as

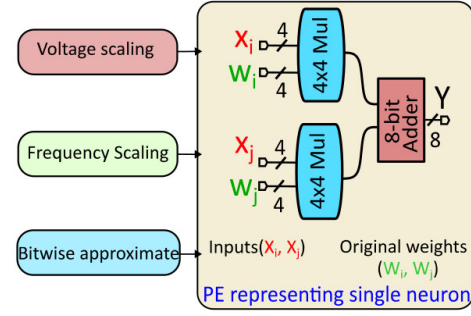


Fig. 2: Design of PE representing single neuron and applied approximate computing techniques.

our test case a relatively simple and readily approximable PE based on the design proposed in [23], consisting of two 4-bit multipliers and an 8-bit adder as can be seen in Fig. 2. This produces a weighted sum value as described in equation 1:

$$Y = x_i \cdot w_i + x_j \cdot w_j \quad (1)$$

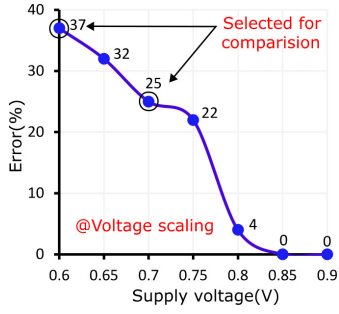
where Y is the weighted sum, x_i , x_j are the inputs and w_i , w_j are the associated weights. Multiple versions of this PE were implemented with varying levels of bitwise approximation from 0 bits (exact design) to 4 bits (50% approximation). More specifically, the bitwise approximation is achieved by replacing all full adders with approximate full adders (this includes those internal to the multipliers as well as the final adder). In all cases the least significant bits are the approximated bits. For VFS evaluation the voltage or frequency of the exact design was varied. As can be seen in Fig 3, to ensure like-for-like comparison two voltage and two frequency points were selected which produce the closest error rate to the 2 and 3 bit bitwise approximation levels. i.e. 25% and 37.5% error respectively. The overall degree of noise in each subsequent comparison varies by no more than 2% between methods. The desired points were determined empirically by stepping through the voltage and frequency ranges until two suitable points were found. The implementation is targeted at 45nm CMOS and simulated using Cadence EDA tools. Again, because the goal here is simply to evaluate the previously untested relative leakage of different approximation methods a simulated test of a single PE suffices to draw some initial conclusions about how these technologies may interact.

IV. CPA ATTACK ON AANN

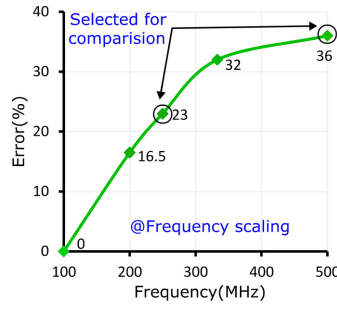
This section describes the weight-stealing CPA attack we use as our test case for leakage evaluation.

A. Attack Scenario

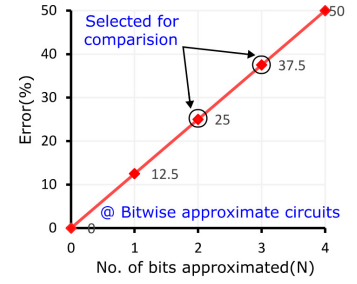
We assume an attack scenario with a relatively strong adversary. The adversary is assumed to have the ability to measure power traces from any internal operation without disrupting that operation. It is further assumed that the adversary can send arbitrary inputs to the ANN at will, and that the adversary has full knowledge of the hardware design - in particular the



(a) Varying supply voltage (voltage scaling technique)



(b) Varying clock frequency (frequency scaling technique)



(c) Increasing number of approximate bits in bitwise approximate circuit

Fig. 3: Percentage of error of PE under different approximation methods

structure of the PEs themselves. The only items of information concealed from the adversary are the specific weights of the deployed network and the original training data. The goal of the adversary is to steal those weight values. For a real world analogy to this threat model, consider a hostile customer who has purchased a naively implemented edge device (therefore, has access to the physical device and to any documentation) and who is attempting to extract a proprietary model from it.

B. Attack Mechanism

For the attack we use a Correlation Power Analysis (CPA) based approach, focusing on the multiply-add operation within the PE. The attack mechanism, visualized in Fig. 4, uses the following steps:

- 1) Two weight values (w_i and w_j) are selected.
- 2) Every possible combination of the input values x_i and x_j are applied in order to the PE using the fixed weights w_i and w_j . The power traces associated with each weighted sum execution is recorded and sampled to form a matrix, A , of size $m \times n$ (where m is the number of input combinations and n is the samples per trace).
- 3) On the other side, the PE algorithm is developed using python based Jupyter note book. An identical input sequence is applied with all possible combinations of weights and associated responses are recorded.
- 4) We use Hamming Distance (HD) to calculate hypothetical power consumption. The HD of two successive outputs of PE is calculated as shown in Eq. 2. This is repeated for each pair of outputs.

$$HD = \text{Hamming Weight}(Y_i \oplus Y_{i+1}) \quad (2)$$

- 5) The obtained hypothetical power consumption values are arranged in a matrix (H) with a size of $m \times k$ where m and k represent the number of inputs and number of weights respectively.
- 6) Finally, each column of H is correlated with all the columns of A , producing a correlation matrix (C) with size $k \times n$. The original weights of PE uniquely show the highest value in the correlation matrix compared to the other keys.

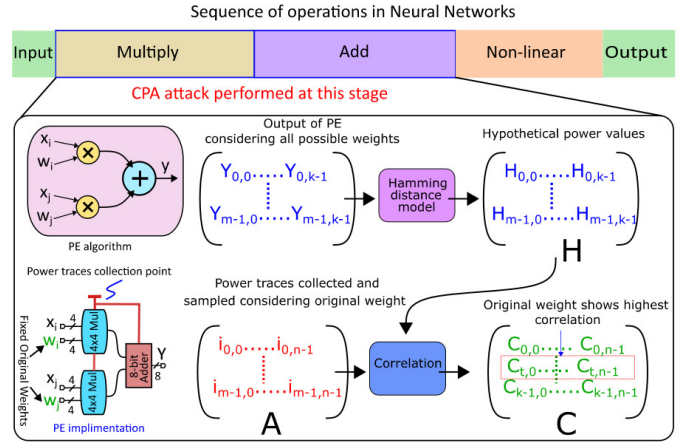


Fig. 4: CPA attack mechanism for retrieving weight information of processing element

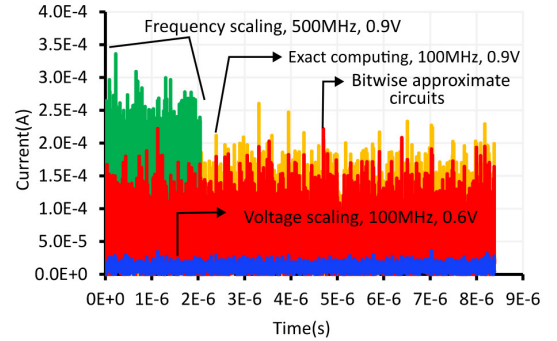


Fig. 5: Sample power traces of PE with different approximate computing techniques (37% in output response).

V. RESULTS AND ANALYSIS

Fig. 5 shows the current measured with each form of approximation during an experiment where $w_i = 1001$, $w_j = 1001$, at a fixed error rate of 37%. This quite clearly illustrates the differences in the approaches from a power analysis perspective. While all three are producing the same degree of error relative to the exact case and working with the same weights and inputs, there is a drastic difference

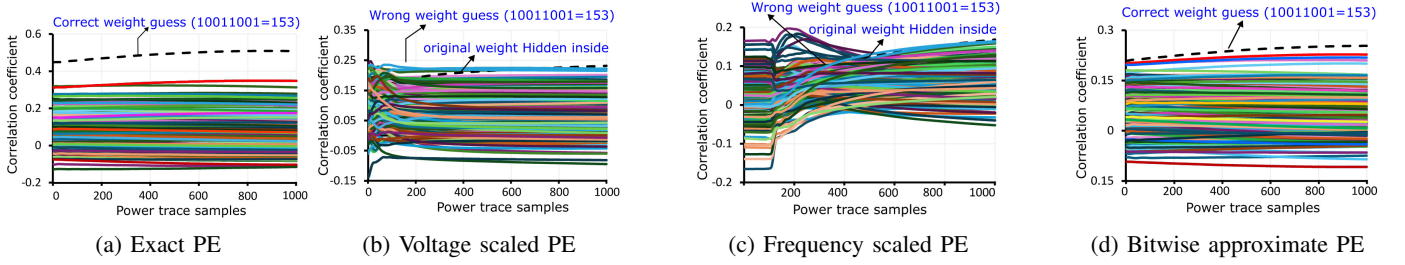


Fig. 6: Correlation coefficients of each technique using 256 power traces

in current. Bitwise approximation remains relatively close to the exact case over time with a moderate reduction in current overall. Frequency scaling is time-compressed and with drastically higher current draw, while voltage scaling exhibits a similarly drastic reduction.

Fig. 6 depicts the correlation coefficient results of all 256 possible weight values for different PE designs using 256 power traces of 1000 samples each. The level of approximation is 25% and the correlation coefficient of the original weight ($w=10011001=153$) is shown as a dotted line. It can be observed that in the exact case the attack is fairly trivial, giving a distinctly higher correlation with the correct weight ($w=153$). When we examine the approximate cases, it can be seen that each approximate PE exhibits a relatively lower correlation coefficient for the original weight compared to the exact PE design. Against voltage and frequency scaling the attack was unable to clearly distinguish the original weight. The attack on the bitwise approximated PE still succeeded, but the distinction between the original weight and the set of other weights is much less than in the exact case.

Clearly, there is a difference between these techniques not only in actual power consumption but also in the amount of useful information in the power trace. To get a clearer picture of what is going on here, we calculated the Signal to Noise Ratio (SNR) from an attack perspective. In this context the SNR is the ratio of the original weight correlation coefficient to the maximum value of the wrong weight guess which is expressed in [24]. i.e. SNR decides the level of difficulty in distinguishing between the original weight and the most likely wrong weight guess. High SNR is better for the attacker and is therefore undesirable. Fig. 7 shows how SNR changes with increasing approximation for each approximate technique. While it can be seen that, as expected from the previous results, all three have lower SNR at higher levels of approximation the behavior is not uniform. Voltage and frequency scaling continue to decline largely proportionally, while bitwise approximation tapers off. In addition, the reduction in SNR is more rapid in the voltage and frequency scaling cases. This accounts for the difference in the example attack shown in Fig. 6, as by the 25% error rate point the SNR of the voltage and frequency scaled PEs is already much lower than the bitwise approximated PE. With higher levels of approximation this gap only widens.

Fig. 8(a-g) shows the measurement to disclosure (MTD) for

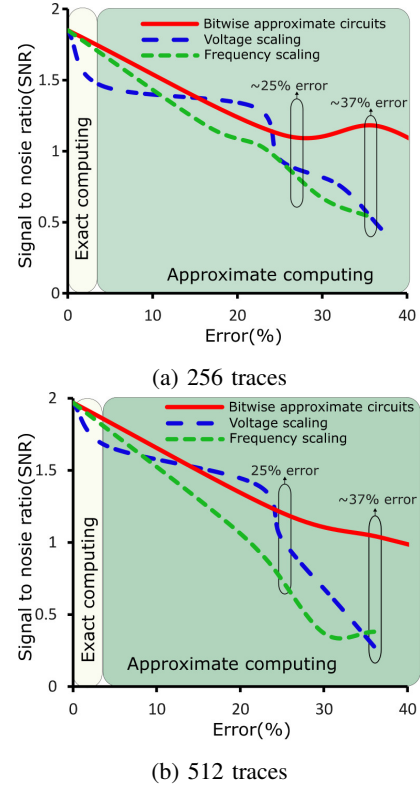


Fig. 7: SNR of PE design varying against output error

each test case. MTD is defined as the minimum number of power traces required to retrieve the original weight information [25], [26]. To calculate this, the attack was repeated using 256, 512, 768, and 1024 traces. The point where the dotted line (original weight) becomes distinct from the general body of potential weights is the MTD. In this test, the baseline value is 48 for an exact PE. Voltage scaling increases this to 570 at 25% error, and above 1024 at 37% error. Frequency scaling likewise increases to roughly 600 at 25% error and somewhere above 1024 at 37% error. As expected from the SNR analysis, bitwise approximation performs better than the exact case but worse than the VFS methods with MTD of 200 at 25% error and roughly 512 at 37% error.

If one considers only information leakage these results show a clear advantage to voltage and frequency scaling. However, the primary design goal of approximation is not security

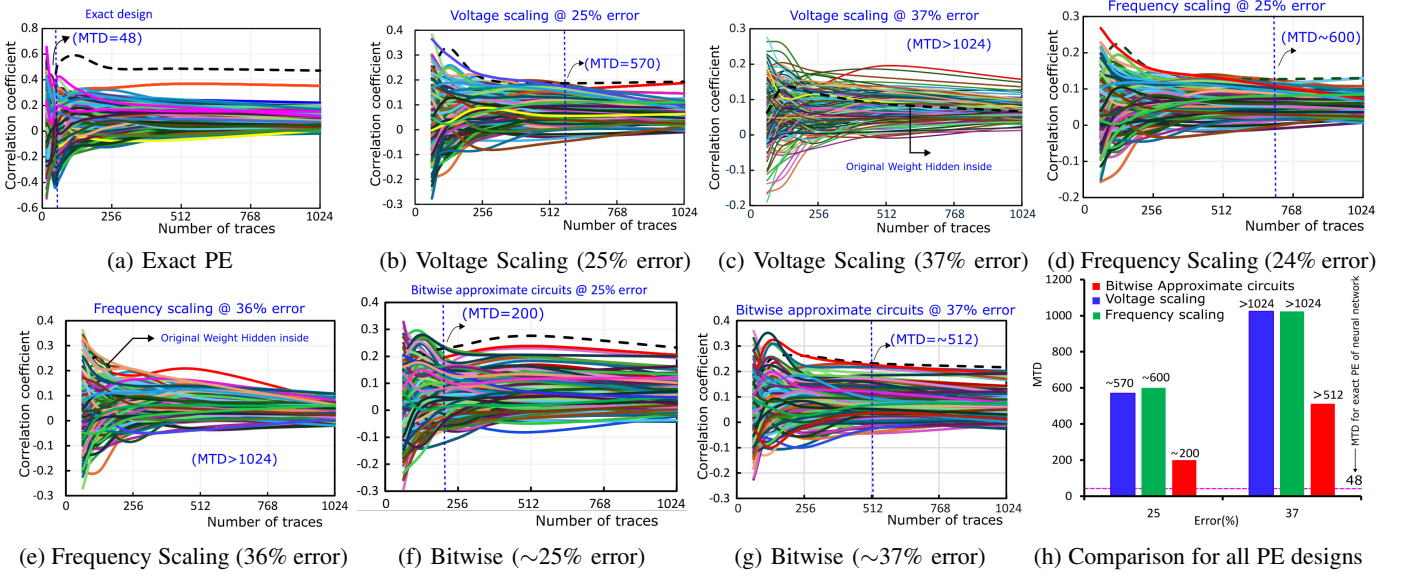


Fig. 8: MTD comparison of approximation method and error rate

TABLE I: Performance comparison of PE designs against CPA

Metrics	Exact Computing	Approximate Computing Techniques				Comments
		Approximation Level	Voltage scaling	Frequency scaling	Bitwise approximation	
MTD	~48	~25%	~570	~600	~200	~4x to ~12x improvement
SPD(fJ)-1	~0.2		~19.5	~11.5	~5	~5x to ~20x improvement
MTD	~48	~37%	>1024	>1024	>512	~11x to ~21x improvement
SPD(fJ)-1	~0.2		~6.9	~6.5	~3.1	~15x to ~35x improvement

but rather efficiency and performance. To better evaluate the performance of each method in this context this work redefines/introduces the security, power, and delay (SPD) metric. The SPD is a figure-of-merit metric that combines the benefits of approximate computing related to security and the system's power consumption and delay [25], [26]. The SPD is the ratio of MTD and the product of power delay as shown in equation 3

$$SPD = \frac{MTD}{Power\ Consumption * Delay} \quad (3)$$

Fig. 9 compares the SPD for each method based on the measurements taken during our experiments. It is evident that the exact PE design shows lower SPD due to the lower MTD, higher power consumption, and higher delay values. Orthogonal to this, approximate computing techniques significantly improve the SPD of PE designs. With a ~25% approximation, PE designs achieve ~20x, ~12x and ~5x higher SPD with voltage scaling, overclocking and approximate circuits respectively when compared with the exact counterpart. Similarly, with a ~37% approximation, PE designs achieve ~35x, ~33x, and 16x higher SPD with voltage scaling, overclocking and approximate circuits respectively. Among the three techniques, the voltage scaling technique achieves higher SPD and approximate circuits techniques show relatively lower SPD. The results have been summarized in Table 1.

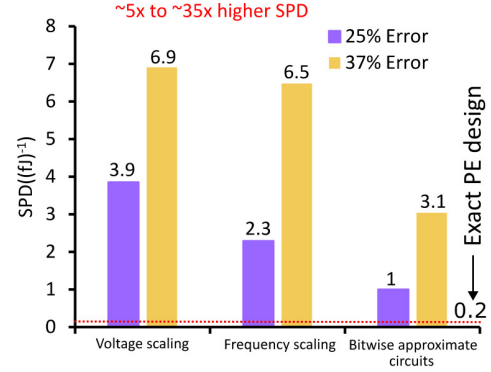


Fig. 9: SPD comparison of PE designs with different approximate computing techniques

VI. CONCLUSION

In this work we have shown that, in an ANN context, approximate computing techniques applied to the fundamental processing elements have a significant and benign impact on information leakage through the power side channel with a minimum increase in attack complexity of $\times 4$ and upwards of $\times 35$ possible. Further, that the magnitude of this effect varies significantly depending on the method and degree of approximation. This provides useful insight for the design of secure and efficient ANNs in the future.

REFERENCES

- [1] X. Hu, L. Liang, S. Li, L. Deng, P. Zuo, Y. Ji, X. Xie, Y. Ding, C. Liu, T. Sherwood, and Y. Xie, “DeepSniffer: A dnn model extraction framework based on learning architectural hints,” in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS ’20*, (New York, NY, USA), p. 385–399, Association for Computing Machinery, 2020.
- [2] K. Patwari, S. M. Hafiz, H. Wang, H. Homayoun, Z. Shafiq, and C.-N. Chuah, “Dnn model architecture fingerprinting attack on cpu-gpu edge devices,” in *2022 IEEE 7th European Symposium on Security and Privacy (EuroSP)*, pp. 337–355, 2022.
- [3] Y. Liu and A. Srivastava, “Ganred: Gan-based reverse engineering of dnns via cache side-channel,” in *Proceedings of the 2020 ACM SIGSAC Conference on Cloud Computing Security Workshop, CCSW’20*, (New York, NY, USA), p. 41–52, Association for Computing Machinery, 2020.
- [4] Ł. Chmielewski and L. Weissbart, “On reverse engineering neural network implementation on gpu,” in *Applied Cryptography and Network Security Workshops: ACNS 2021 Satellite Workshops, AIBlock, AIHWS, AIoTS, CIMSS, Cloud S&P, SCI, SecMT, and SiMLA, Kamakura, Japan, June 21–24, 2021, Proceedings*, pp. 96–113, Springer, 2021.
- [5] S. Tian, S. Moini, A. Wolnikowski, D. Holcomb, R. Tessier, and J. Szefer, “Remote power attacks on the versatile tensor accelerator in multi-tenant fpgas,” in *2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pp. 242–246, IEEE, 2021.
- [6] Y. Zhang, R. Yasaei, H. Chen, Z. Li, and M. A. Al Faruque, “Stealing neural network structure through remote fpga side-channel analysis,” *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 4377–4388, 2021.
- [7] V. Meyers, D. Gnad, and M. Tahoori, “Reverse engineering neural network folding with remote fpga power analysis,” in *2022 IEEE 30th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pp. 1–10, IEEE, 2022.
- [8] X. Yan, X. Lou, G. Xu, H. Qiu, S. Guo, C. H. Chang, and T. Zhang, “Mercury: An automated remote side-channel attack to nvidia deep learning accelerator,” in *2023 International Conference on Field Programmable Technology (ICFPT)*, pp. 188–197, 2023.
- [9] L. Batina, S. Bhasin, D. Jap, and S. Picek, “CSI NN: Reverse engineering of neural network architectures through electromagnetic side channel,” in *28th USENIX Security Symposium (USENIX Security 19)*, (Santa Clara, CA), pp. 515–532, USENIX Association, Aug. 2019.
- [10] S. Maji, U. Banerjee, and A. P. Chandrakasan, “Leaky nets: Recovering embedded neural network models and inputs through simple power and timing side-channels—attacks and defenses,” *IEEE Internet of Things Journal*, vol. 8, no. 15, pp. 12079–12092, 2021.
- [11] K. Yoshida, T. Kubota, M. Shiozaki, and T. Fujino, “Model-extraction attack against fpga-dnn accelerator utilizing correlation electromagnetic analysis,” in *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pp. 318–318, 2019.
- [12] K. Yoshida, T. Kubota, S. Okura, M. Shiozaki, and T. Fujino, “Model reverse-engineering attack using correlation power analysis against systolic array based neural network accelerator,” in *2020 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5, 2020.
- [13] A. Dubey, R. Cammarota, and A. Aysu, “Maskednet: The first hardware inference engine aiming power side-channel protection,” in *2020 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pp. 197–208, 2020.
- [14] M. Fredrikson, E. Lantz, S. Jha, S. Lin, D. Page, and T. Ristenpart, “Privacy in pharmacogenetics: An {End-to-End} case study of personalized warfarin dosing,” in *23rd USENIX security symposium (USENIX Security 14)*, pp. 17–32, 2014.
- [15] Z. Chen, M. Tang, and J. Li, “Inversion attacks against cnn models based on timing attack,” *Security and Communication Networks*, vol. 2022, no. 1, p. 6285909, 2022.
- [16] L. Wei, B. Luo, Y. Li, Y. Liu, and Q. Xu, “I know what you see: Power side-channel attack on convolutional neural network accelerators,” in *Proceedings of the 34th Annual Computer Security Applications Conference, ACSAC ’18*, (New York, NY, USA), p. 393–406, Association for Computing Machinery, 2018.
- [17] K. Ganju, Q. Wang, W. Yang, C. A. Gunter, and N. Borisov, “Property inference attacks on fully connected neural networks using permutation invariant representations,” in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, pp. 619–633, 2018.
- [18] M. Chase, E. Ghosh, and S. Mahloujifar, “Property inference from poisoning,” *arXiv preprint arXiv:2101.11073*, 2021.
- [19] C. Fu, X. Zhang, S. Ji, J. Chen, J. Wu, S. Guo, J. Zhou, A. X. Liu, and T. Wang, “Label inference attacks against vertical federated learning,” in *31st USENIX security symposium (USENIX Security 22)*, pp. 1397–1414, 2022.
- [20] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, “Exploiting unintended feature leakage in collaborative learning,” in *2019 IEEE symposium on security and privacy (SP)*, pp. 691–706, IEEE, 2019.
- [21] G. Armeniakos, G. Zervakis, D. Soudris, and J. Henkel, “Hardware approximate techniques for deep neural network accelerators: A survey,” *ACM Comput. Surv.*, vol. 55, Nov. 2022.
- [22] F. Regazzoni and I. Polian, “Side channel attacks vs approximate computing,” in *Proceedings of the 2020 on Great Lakes Symposium on VLSI, GLSVLSI ’20*, (New York, NY, USA), p. 321–326, Association for Computing Machinery, 2020.
- [23] A. Balaji, S. Ullah, A. Das, and A. Kumar, “Design methodology for embedded approximate artificial neural networks,” in *Proceedings of the 2019 Great Lakes Symposium on VLSI, GLSVLSI ’19*, (New York, NY, USA), p. 489–494, Association for Computing Machinery, 2019.
- [24] S. D. Kumar, H. Thapliyal, and A. Mohammad, “Finsal: A novel finfet based secure adiabatic logic for energy-efficient and dpa resistant iot devices,” in *2016 IEEE International Conference on Rebooting Computing (ICRC)*, pp. 1–8, 2016.
- [25] I. M. Delgado-Lozano, E. Tena-Sánchez, J. NÚñez, and A. J. Acosta, “Design and analysis of secure emerging crypto-hardware using hyperfet devices,” *IEEE Transactions on Emerging Topics in Computing*, vol. 9, no. 2, pp. 787–796, 2021.
- [26] E. Tena-Sánchez and A. J. Acosta, “Logic minimization and wide fan-in issues in dpl-based cryptocircuits against power analysis attacks,” *International Journal of Circuit Theory and Applications*, vol. 47, no. 2, pp. 238–253, 2019.